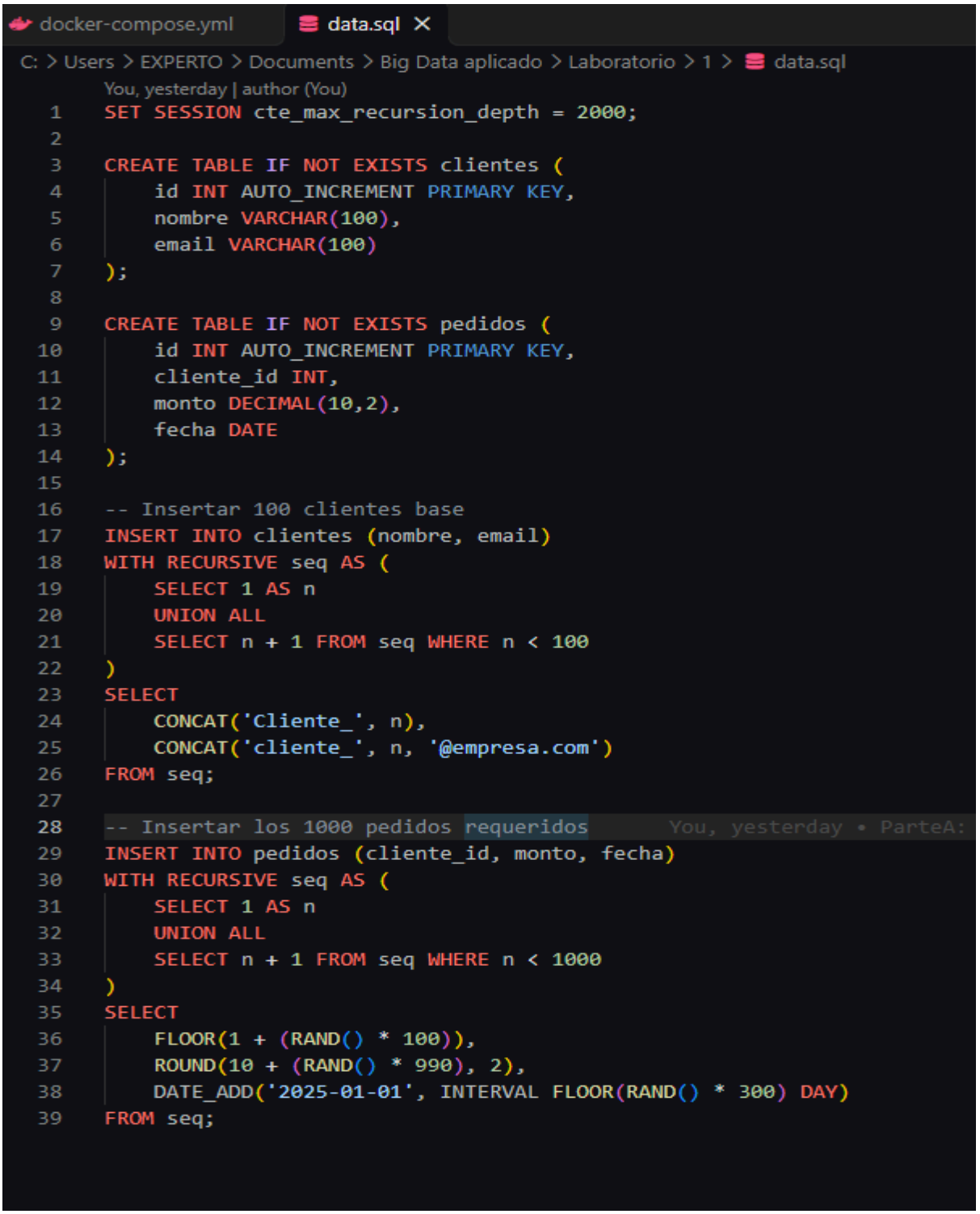
A1

1-Primero, creamos la BD con el siguiente codigo



```
docker-compose.yml   data.sql
C: > Users > EXPERTO > Documents > Big Data aplicado > Laboratorio > 1 > data.sql
      You, yesterday | author (You)
1   SET SESSION cte_max_recursion_depth = 2000;
2
3   CREATE TABLE IF NOT EXISTS clientes (
4       id INT AUTO_INCREMENT PRIMARY KEY,
5       nombre VARCHAR(100),
6       email VARCHAR(100)
7   );
8
9   CREATE TABLE IF NOT EXISTS pedidos (
10      id INT AUTO_INCREMENT PRIMARY KEY,
11      cliente_id INT,
12      monto DECIMAL(10,2),
13      fecha DATE
14  );
15
16  -- Insertar 100 clientes base
17  INSERT INTO clientes (nombre, email)
18  WITH RECURSIVE seq AS (
19      SELECT 1 AS n
20      UNION ALL
21      SELECT n + 1 FROM seq WHERE n < 100
22  )
23  SELECT
24      CONCAT('Cliente_', n),
25      CONCAT('cliente_', n, '@empresa.com')
26  FROM seq;
27
28  -- Insertar los 1000 pedidos requeridos      You, yesterday • ParteA:
29  INSERT INTO pedidos (cliente_id, monto, fecha)
30  WITH RECURSIVE seq AS (
31      SELECT 1 AS n
32      UNION ALL
33      SELECT n + 1 FROM seq WHERE n < 1000
34  )
35  SELECT
36      FLOOR(1 + (RAND() * 100)),
37      ROUND(10 + (RAND() * 990), 2),
38      DATE_ADD('2025-01-01', INTERVAL FLOOR(RAND() * 300) DAY)
39  FROM seq;
```

2-Después, debemos ir a la ruta donde tenemos docker-compose y data.sql, en mi caso C:\Users\EXPERTO\Documents\Big Data aplicado

En está ruta lanzamos el comando con las credenciales dadas por el ejercicio
Get-Content data.sql | docker exec -i mysql-server mysql -u sqoopuser -p'sqooppassword' ventas_db
Con esto haremos que el ordenador lea el contenido de nuestro archivo sql y ordene al contenedor mysql-server de docker a recibir los datos usando el usuario y contraseña correspondientes.

3-docker exec -it mysql-server mysql -u sqoopuser -p'sqooppassword' -e "USE ventas_db; SELECT COUNT(*) FROM pedidos;"
Con este comando podemos pedir que se cuenten todos los registros del servidor y nos da 1000, con eso comprobamos que ha absorbido los 1000 registros correctamente.

```
PS C:\Users\EXPERTO\Documents\Big Data aplicado\Laboratorio\1> docker exec -it mysql-server mysql -u sqoopuser -p'sqooppassword' -e "USE ventas_db; SELECT COUNT(*) AS total_pedidos FROM pedidos;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+--------------+
| total_pedidos |
+--------------+
|         1000 |
+--------------+
```

A2

4-Con el comando docker exec -it namenode bash entramos al contenedor donde tenemos las herramientas necesarias para el ejercicio(Hadoop y Scoop).
Con todo esto, podemos ejecutar la ingesta, tal como está en el ejercicio

```
[root@namenode hadoop]# sqoop import \
> --connect jdbc:mysql://localhost/ventas_db \
> --username sqoopuser \
> --password 'sqooppassword' \
> --table pedidos \
> --target-dir /user/iabd_admin/ingesta/ventas/pedidos \
> --m 1
```

Al final se puede ver como la ingesta se hace correctamente:

```
2026-11-05 18:25:58 INFO  ImportJobBase:188 - Transferred 24.123 KB in 31.755 seconds (777.8926 bytes/sec)
2026-11-05 18:25:58 INFO  ImportJobBase:190 - Retrieved 1000 records.
```

A3

Comprobamos que el directorio se ha creado y existe la carpeta pedidos:

```
[root@namenode hadoop]# hdfs dfs -ls /user/iabd_admin/ingesta/ventas
Found 1 items
drwxr-xr-x   - hadoop supergroup          0 2026-11-05 18:25 /user/iabd_admin/ingesta/ventas/pedidos
```

A continuación vemos los ficheros que ha creado dentro de dicha carpeta:

```
[root@namenode hadoop]# hdfs dfs -ls /user/iabd_admin/ingesta/ventas/pedidos
Found 2 items
-rw-r--r--   3 hadoop supergroup          0 2026-11-05 18:25 /user/iabd_admin/ingesta/ventas/pedidos/_SUCCESS
-rw-r--r--   3 hadoop supergroup      24702 2026-11-05 18:25 /user/iabd_admin/ingesta/ventas/pedidos/part-m-00000
```

Finalmente, leemos los datos desde HDFS:

```
[root@namenode hadoop]# hdfs dfs -cat /user/iabd_admin/ingesta/ventas/pedidos/part-m-00000
1,38,85.33,2025-03-19
2,6,513.63,2025-04-23
3,36,646.82,2025-02-16
4,84,727.37,2025-02-03
5,39,605.88,2025-09-11
6,43,591.15,2025-07-17
7,54,713.47,2025-10-09
8,57,996.75,2025-03-30
9,49,555.83,2025-03-30
10,82,204.29,2025-06-10
11,9,815.52,2025-09-03
12,65,795.56,2025-01-06
13,72,516.15,2025-05-05
```

B2

1-Primero creamos el directorio para nuestras configuraciones:

```
[root@namenode hadoop]# mkdir -p ~/flume_configs
[root@namenode hadoop]# cd ~/flume_configs
[root@namenode flume_configs]#
```

2-Instalamos nano:

```
[root@namenode flume_configs]# yum install -y nano
```

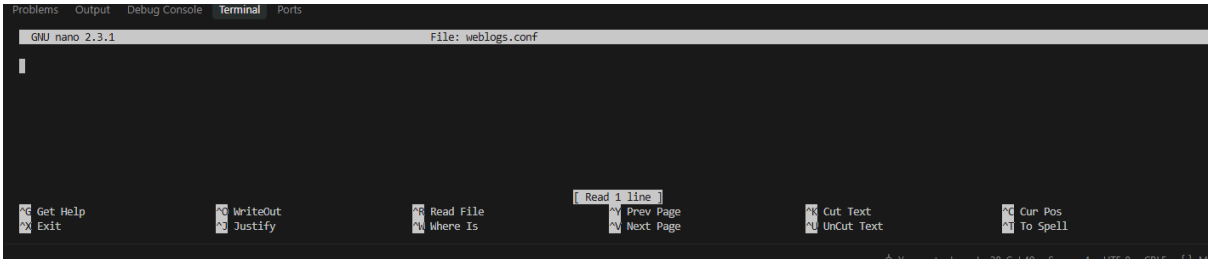
```
Installed:
  nano.x86_64 0:2.3.1-10.el7

Complete!
```

3-Ahora creamos el fichero de configuración:

```
[root@namenode flume_configs]# nano weblogs.conf
```

Nos saldrá una interfaz como esta:



4-Copiamos y pegamos el código exactamente como lo da el ejercicio y salimos del nano:

B3

1-Nos aseguramos de que el spooldir que hemos definido exista:

```
[root@namenode flume_configs]# mkdir -p /home/iabd_admin/flume_ingest
[root@namenode flume_configs]#
```

2-Con el comando cd /opt
curl -O https://archive.apache.org/dist/flume/1.9.0/apache-flume-1.9.0-bin.tar.gz
tar -xzf apache-flume-1.9.0-bin.tar.gz
mv apache-flume-1.9.0-bin flume

instalamos flume en nuestro docker.
Comprobamos que la instalación ha salido bien:

```
[root@namenode opt]# flume-ng version
Flume 1.9.0
Source code repository: https://git-wip-us.apache.org/repos/asf/flume.git
Revision: d4fcab4f501d41597bc616921329a4339f73585e
Compiled by fszabo on Mon Dec 17 20:45:25 CET 2018
From source with checksum 35db629a3bda49d23e9b3690c80737f9
[root@namenode opt]#
```

B4

3-Lanzamos Flume en la primera terminal, como podemos ver, se queda en Streaming, es decir “escuchando en tiempo real indefinidamente”:

```
[root@namenode flume_configs]# flume-ng agent \
> --conf ~/flume_configs/ \
> --conf-file ~/flume_configs/weblogs.conf \
> --name agent \
> -Dflume.root.logger=INFO,console
Info: Including Hive libraries found via () for Hive access
+ exec /usr/lib/jvm/jre//bin/java -Xmx20m -Dflume.root.logger=INFO,console -cp '/root/flume_configs:/opt/flume/lib/*:/lib/*' -Djava.library.path= org.apache.flume.node.Application --conf-file /r
me_configs/weblogs.conf --name agent
log4j:WARN No appenders could be found for logger (org.apache.flume.util.SSLUtil).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
```

4-Ahora debemos crear otra terminal.
En esta segunda terminal debemos simular que el servidor web genera logs. Crearemos un fichero y lo moveremos al directorio que Flume está vigilando.
Terminal 2:

```
[root@namenode hadoop]# echo "192.168.1.1 - GET /index.html" > ~/log_temp.txt
[root@namenode hadoop]# echo "192.168.1.5 - POST /login.php" >> ~/log_temp.txt
[root@namenode hadoop]# mv ~/log_temp.txt /home/iabd_admin/flume_ingest/log_file_03.txt
```

En la terminal 1 podremos ver esto en directo.
Podemos verificar que se ha hecho la ingesta usando este comando en la terminal 2 hdfs dfs -ls /user/iabd_admin/ingesta/weblogs

```
[root@namenode hadoop]# hdfs dfs -ls /user/iabd_admin/ingesta/weblogs
Found 4 items
-rw-r--r--   3 root supergroup         68 2026-11-05 19:21 /user/iabd_admin/ingesta/weblogs/FlumeData.1793906424913
-rw-r--r--   3 root supergroup         60 2026-11-05 19:23 /user/iabd_admin/ingesta/weblogs/FlumeData.1793906539552
-rw-r--r--   3 root supergroup          1 2026-11-05 19:29 /user/iabd_admin/ingesta/weblogs/FlumeData.1793906906273
-rw-r--r--   3 root supergroup         60 2026-11-05 19:32 /user/iabd_admin/ingesta/weblogs/FlumeData.1793907104217
[root@namenode hadoop]#
```

5-Si queremos generar más datos, solo debemos repetir el proceso
Creamos un nuevo fichero con este comando:

```
[root@namenode hadoop]# echo "192.168.1.100 - GET /final_test.html" > ~/log_temp.txt
[root@namenode hadoop]# echo "192.168.1.101 - GET /success.php" >> ~/log_temp.txt
[root@namenode hadoop]# mv ~/log_temp.txt /home/iabd_admin/flume_ingest/log_file_02.txt
```

Si volvemos a revisar podemos comprobar que se crea correctamente:

```
[root@namenode hadoop]# hdfs dfs -ls /user/iabd_admin/ingesta/weblogs
Found 5 items
-rw-r--r--   3 root supergroup         68 2026-11-05 19:21 /user/iabd_admin/ingesta/weblogs/FlumeData.1793906424913
-rw-r--r--   3 root supergroup         60 2026-11-05 19:23 /user/iabd_admin/ingesta/weblogs/FlumeData.1793906539552
-rw-r--r--   3 root supergroup          1 2026-11-05 19:29 /user/iabd_admin/ingesta/weblogs/FlumeData.1793906906273
-rw-r--r--   3 root supergroup         60 2026-11-05 19:32 /user/iabd_admin/ingesta/weblogs/FlumeData.1793907104217
-rw-r--r--   3 root supergroup         70 2026-11-05 19:38 /user/iabd_admin/ingesta/weblogs/FlumeData.1793907509305.tmp
[root@namenode hadoop]#
```

DISCUSIÓN Y CIERRE:

- ¿Por qué no usamos Sqoop para los logs? ¿Qué pasaría?

  Porque Sqoop solo sirve para ingestar datos de forma puntual, los logs requieren ingestarse en tiempo real.

  Si usaramos Sqoop habría que ingestar manualmente cada log, mientras que con Flume se ingestan en tiempo real al instante de crearse. Para este ejercicio hemos creado los logs manualmente a modo de prueba y ya el propio Flume se encarga de trabajar con ellos, sin intervención nuestra.

- ¿Por qué no usamos Flume para la base de datos? ¿Se podría?

  Es posible usarlo, sin embargo, no es lo ideal, puesto que Sqoop cuenta con plugins y herramientas para trabajar con BDs. Por ejemplo, Sqoop divide automáticamente las tablas grandes en rangos mediante la clave primaria y lanza múltiples *mappers* en paralelo para acelerar la extracción.

- En Flume, usamos un memory channel. ¿Qué riesgo tiene esto en un entorno de producción real?

  El mayor riesgo es que en caso de un fallo total, deberemos parar el Flume, por lo que **todos** los datos almacenados en el canal que aún no se hayan escrito en HDFS se pierden definitivamente.